📘 Student

Day 1 — Foundation Track

Student Workbook · Labs 00–12

Copilot Studio Workshop · Generated April 28, 2026

Day 1 Foundation Guide

Day 1 is your foundation day. The goal is to help every participant leave with a working understanding of how Copilot Studio agents are structured, built, tested, and published — using the Woodgrove Bank customer service scenario.

Schedule overview

Use this as your pacing map for the day:

Time block	Focus
Morning kickoff	Sign-in, environment checks, workshop framing
Morning build	Agent concepts and Copilot Studio fundamentals
Late morning	Declarative patterns, solutions, and prebuilt agent accelerators
Early afternoon	Customer service agent grounded in banking data and solution structure
Mid afternoon	Topics, Adaptive Cards, flows, and triggers
Late afternoon	Publish review, licensing and Copilot Credits discussion, recap, and Day 2 readiness

Prep reminders

Before the first module starts, confirm:

- You can sign in with the correct Microsoft 365 account.
- You can access Copilot Studio in the target environment.
- You can access SharePoint content used for grounding (Woodgrove Bank customer accounts and service requests).
- You know who to ask if licensing, permissions, or environment access is blocked.

Start here if you need setup help:

- Lab 00 - Environment Setup

Module map

Work through the day in this order:

1. Understand what agents are and where Copilot Studio fits.

2. Learn the core building blocks of agent behavior and grounding.
3. Explore declarative patterns and when they fit the scenario.
4. Package work in solutions and review prebuilt agents as reusable accelerators.
5. Build the Woodgrove Customer Service Agent with knowledge grounded in banking data.
6. Add interaction patterns with topics, Adaptive Cards, flows, and triggers.
7. Review publishing, Copilot Credits, ROI talking points, and readiness for the next day.

Helpful lab links:

- Lab 01 - Intro to Agents
- Lab 02 - Copilot Studio Fundamentals
- Lab 03 - Declarative Agents
- Lab 04 - Solutions
- Lab 05 - Prebuilt Agents
- Lab 06 - Build the Customer Service Agent
- Lab 07 - Topics and Triggers
- Lab 08 - Adaptive Cards
- Lab 09 - Agent Flows
- Lab 10 - Event Triggers
- Lab 11 - Publish Agent
- Lab 12 - Licensing and Copilot Credits

Expected outputs

By the end of Day 1, you should have:

- A clear understanding of the difference between core agent concepts and implementation choices.
- Experience creating or modifying an agent in Copilot Studio.
- A grounded agent that can respond from Woodgrove Bank SharePoint data, including customer accounts and service requests.
- Exposure to solution packaging, prebuilt agent patterns, and the Day 1 Copilot Credits and ROI close.
- Exposure to topics, cards, flows, triggers, and publishing workflow.
- A clear sense of whether you are ready for Day 2 advanced modules.

Lab flow

Module 1: Orientation and concepts

Focus on the business value of agents in a banking context, not just the definitions. Pay attention to how grounding, orchestration, and user interaction all fit together for customer service scenarios.

Module 2: Foundations in Copilot Studio

Use this section to learn where the main authoring surfaces live and what each one controls. If the interface is new to you, prioritize understanding the flow over memorizing every menu.

Module 3: Declarative patterns, solutions, and prebuilt agents

Observe how simple prompt-driven experiences differ from more tailored agents. Use the solutions and prebuilt-agent labs to understand what can be packaged, reused, and accelerated before you start from scratch.

Module 4: Customer service agent and grounding

Build toward a working agent experience by creating the Woodgrove Customer Service Agent and attaching banking knowledge that supports grounded responses about customer accounts and service requests.

Module 5: Topics, cards, flows, and triggers

Use this module to connect conversation with action. Focus on what input the agent collects, what process it triggers, and how success is validated in a customer service workflow.

Module 6: Publish and reflect

Finish by understanding how a workshop prototype differs from a production-ready asset. Watch for environment, access, and governance considerations called out by your facilitator.

Module 7: Copilot Credits, ROI, and Day 2 prep

Close by reviewing Copilot Credits, user licensing versus capacity consumption, and the rollout or ROI questions your team would need to answer before scaling the customer service scenario.

> **Tip:** If you fall behind, aim to complete one strong end-to-end path rather than skimming every screen without a working result.

Audience notes

- [Maker] Focus on clarity of instructions, prompts, and user experience.

- [IT Pro] Focus on environment setup, access, publishing permissions, and governance implications.
- [Developer] Focus on solution structure, extensibility, and patterns you would automate or formalize later.

End-of-day recap

At the close of Day 1, make sure you can answer these questions:

- What makes an agent grounded and useful for banking customer service?
- Which Copilot Studio features did you use directly today?
- Which parts of the build require environment or governance planning?
- What Copilot Credits or readiness questions still need answers before production rollout?
- Are you ready to continue into the enterprise Loan Processing Agent scenario tomorrow?

Day 1 completion guidance

Use the day's completed modules as your checkpoint toward Day 1 completion. If you still feel uncertain about the foundations, revisit the linked labs before starting Day 2.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 00 — Environment Setup

⏱️ Estimated time: 45 min

Overview

In this lab, you will prepare the shared Woodgrove Bank workshop scenario used throughout Day 1. You will verify participant prerequisites, activate Copilot Studio access, select the correct Power Platform environment, and create the SharePoint site and lists that later labs use for grounding, Product Applications, and automated ticket handling.

> **Note:** If your organization already provides a Microsoft 365 tenant, a Power Platform environment, and Copilot Studio access, complete the validation steps first and then skip directly to the SharePoint setup section.

Copilot Studio home page with environment selector and create options

Prerequisites

1. Confirm you have a work or school account that can sign in to Microsoft 365. Personal Microsoft accounts are not supported.
2. Confirm you can open `https://copilotstudio.microsoft.com` in Microsoft Edge or Google Chrome.
3. [Maker] Confirm you can create resources in a Power Platform environment.
4. [IT Pro] Confirm self-service trial sign-up is allowed in the tenant, or have an approved alternative environment ready.
5. [Developer] Have a second browser profile or InPrivate window available so you can test sign-in prompts without affecting your daily tenant session.
6. Keep this lab open and record the values you use for the tenant name, environment name, and SharePoint site URL.

Step-by-Step Instructions

Step 1 — Verify tenant access

1. Open a browser tab, go to `https://www.office.com`, and select **Sign in**.
2. Enter your workshop account in the **Email, phone, or Skype** field, select **Next**, enter your password, and complete any multifactor prompt.
3. Select the **App launcher** in the upper-left corner and confirm you can see Microsoft 365 apps such as **Teams**, **SharePoint**, and **Power Apps**.
4. If you are using a new tenant, open **Admin** from the app launcher, confirm the tenant domain shown in the header, and note it for later validation.

Step 2 — Start or confirm Copilot Studio access

1. In a new browser tab, go to `https://aka.ms/TryCopilotStudio` and sign in with the same workshop account.
2. Select **Start free trial** if the page offers a trial, or select **Open Copilot Studio** if access is already provisioned.
3. Wait for the confirmation screen, then open `https://copilotstudio.microsoft.com`.
4. In Copilot Studio, confirm the page loads without an access error and that the top bar shows your signed-in account.

> **Tip:** If you receive a self-service sign-up error, capture the exact message and switch to the troubleshooting section before continuing.

Step 3 — Create or confirm a Power Platform development environment

1. Go to `https://aka.ms/PowerAppsDevPlan` and sign in with the same workshop account.
2. Select **Start free** on the **Power Apps Developer Plan** page if you do not already have a developer environment.
3. After the sign-up completes, open `https://make.powerapps.com`.
4. In the environment selector in the upper-right corner, open the dropdown and select your developer environment, such as **Adele Vance's environment**.
5. Return to `https://copilotstudio.microsoft.com`, open the environment selector, and select the same developer environment.
6. Record the exact environment name because you will reuse it in every Day 1 lab.

Step 4 — Enable authoring and publishing permissions

1. [IT Pro] Open `https://admin.microsoft.com` and select **Teams & groups** > **Active teams & groups**. Alternatively, use `https://entra.microsoft.com` > **Groups** > **All groups** > **New group** if you manage groups through Microsoft Entra ID. Commercial tenants may also use `https://admin.cloud.microsoft` as an alternative entry point.

2. Select the **Security groups** tab, select **Add a security group**, and create a group named `WoodgroveBankAgents`.
3. Open the new `WoodgroveBankAgents` group, select **Members**, select **View all and manage members**, and add the participant accounts.
4. Open `https://admin.powerplatform.com`, select **Manage**, then select **Tenant settings**.
5. Scroll to **Copilot Studio authors**, select the **Edit** pencil, choose the `WoodgroveBankAgents` security group, and select **Save**.
6. [Maker] Sign out and back in to Copilot Studio if the new permissions do not appear immediately.

> **Warning:** Trial environments may allow authoring but block publishing. Complete this step before Lab 11 if you want participants to finish the Teams publishing exercise.

Step 4b — Confirm Copilot Studio credits are available

1. [IT Pro] Open `https://admin.powerplatform.com` and select your workshop environment.
2. In the environment details, look for a **Billing** or **Licensing** section.
3. Confirm that one of the following is in place:
   - **Pay-as-you-go:** An Azure subscription is linked to the environment for usage-based billing. To set this up, select **Billing** > **Link Azure subscription**, choose your subscription, and select **Save**.
   - **Capacity pack:** A Copilot Studio capacity pack is assigned to the tenant and the environment has access to the pooled credits.
   - **Trial:** A Copilot Studio trial is active and has not expired.
4. To verify, open Copilot Studio, create a quick test agent, and send a message in the **Test** pane. If you receive a response without a capacity or licensing error, credits are working.

> **Warning:** Without Copilot Studio credits, agent authoring may work but advanced actions, connectors, triggers, and publishing will fail. Set up billing before continuing to Lab 06.

Step 5 — Create the Woodgrove Bank SharePoint site

1. In Microsoft 365, select the **App launcher**, then select **SharePoint**.
2. On the SharePoint start page, select **+ Create site**.
3. In the site type dialog, select **Team site**.
4. In the template gallery, select **IT help desk** and then select **Use template**.
5. In the **Site name** field, enter `Woodgrove Bank`.
6. In the **Site description** field, enter `Copilot Studio workshop data source`.

7. In the **Site address** field, enter `WoodgroveBank` or the nearest available variation.
8. Leave the **Language** field set to **English** and select **Create site**.
9. On the **Add members** screen, add workshop participants if required, then select **Finish**.
10. When the site opens, copy the site URL from the browser address bar and save it in your notes.

SharePoint team site creation form with Woodgrove Bank values

Step 6 — Prepare the Customer Accounts list for later labs

1. In the **Woodgrove Bank** site, select **Site contents** and open the **Customer Accounts** list.
2. Scroll to the far right of the list header, select **+ Add column**, choose **Hyperlink**, enter `Image` in the **Name** field, and select **Save**.
3. Select **New** and create at least four sample items using the values below, keeping **Status** set to `Available` for at least three items.
4. Enter values such as `Surface Laptop 13`, `Surface Laptop 15`, `Surface Pro 12`, and `Surface Studio` in the **Title** field.
5. Populate the remaining fields with realistic values for **Manufacturer**, **Model**, **Asset Type**, **Color**, **Serial Number**, **Purchase Date**, **Purchase Price**, and **Order #**.
6. Open the **Status** column settings and confirm the choices include `Available` and `Requested`. Add `Requested` if it is missing, because Lab 09 uses this value when processing account request flows.
7. If your tenant allows public image links, paste a valid PNG or JPG URL into the new **Image** column; otherwise leave the field blank.
8. Save each item and confirm the list displays the rows.

> **Tip:** If your SharePoint template did not provision a **Customer Accounts** list or the columns differ from the steps above, select **New** > **List** > **Blank list**, name it `Customer Accounts`, and add the columns manually: Title (default), Manufacturer (Text), Model (Text), Asset Type (Text), Color (Text), Serial Number (Text), Purchase Date (Date), Purchase Price (Currency), Order # (Text), Status (Choice with values Available, Requested, Retired), and Image (Hyperlink).

Step 7 — Prepare the Service Requests list for later labs

1. Return to **Site contents** and open the **Service Requests** list.
2. Select **New** and create a sample ticket with **Title** set to `VPN connection issue`.
3. Enter `User cannot connect after password change` in the **Description** field.
4. Set **Priority** to `Normal`, save the item, and confirm the ticket appears in the list.
5. Keep the **Service Requests** list available because Lab 10 uses it for event triggers.

Validation

1. Open `https://copilotstudio.microsoft.com` and confirm the correct workshop environment appears in the environment selector.
2. Confirm the **Woodgrove Bank** SharePoint site opens successfully from the saved site URL.
3. Confirm the **Customer Accounts** list contains at least four items and the **Image** column is visible.
4. Confirm the **Service Requests** list contains at least one sample ticket.
5. Record these four values in your notes: tenant domain, environment name, SharePoint site URL, and one sample account title.

Troubleshooting

> **Tip:** If Copilot Studio opens in the wrong environment, select the environment name in the upper-right corner and switch back to your developer environment before creating any agents.

> **Tip:** If the Power Apps Developer Plan does not create an environment, wait five minutes, refresh `https://make.powerapps.com`, and check the environment selector again.

> **Tip:** If SharePoint template provisioning is slow, keep the page open until the site home page loads; creating the site twice often causes duplicate cleanup work.

> **Warning:** If you cannot start the Copilot Studio trial because self-service sign-up is disabled, escalate to your tenant admin and ask for Copilot Studio access or a pre-provisioned environment.

Facilitator Notes

1. Complete this pre-event checklist at least one business day before delivery: verify tenant access, verify Copilot Studio trial or paid capacity, verify Power Platform admin access, verify a publishing-capable environment, verify Copilot Studio credits (pay-as-you-go Azure billing, capacity pack, or active trial), and verify SharePoint site creation permissions.
2. Pre-create at least one backup environment and one backup SharePoint site so participants can continue if tenant provisioning fails.

3. Decide whether participants will use their own trial tenants or a shared customer tenant, then communicate the login pattern before the event starts.
4. Prepare a short slide or whiteboard note with the standard values `Woodgrove Bank`, `Woodgrove Customer Service Agent`, and `WoodgroveBankAgents` so naming stays consistent.
5. Ask one [IT Pro] helper to stay available during the first 30 minutes for licensing, MFA, and security-group issues.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 01 — Introduction to Agents

⏱️ Estimated time: 20 min

Overview

This lab establishes the concepts used in the rest of the workshop: conversational AI, large language models, retrieval-augmented generation, conversational agents, and autonomous agents. You will use the Woodgrove Bank scenario to decide when each pattern is the right fit before you build anything in Copilot Studio.

> **Note:** This is a discussion-first lab, but it still has concrete participant tasks so you can validate the concepts before moving into hands-on authoring.

Workshop slide showing conversational versus autonomous agents

Prerequisites

1. Complete Lab 00 and confirm you can sign in to Copilot Studio.
2. Open a note-taking app such as **OneNote** or **Notepad** for your answers.
3. Join the instructor-led discussion channel in **Microsoft Teams** if your cohort is using one.

Step-by-Step Instructions

Step 1 — Review the Woodgrove Bank scenario

1. Open this lab and read the **Overview** section aloud or silently.
2. In your note-taking app, create three headings: `Questions`, `Tasks`, and `Triggers`.
3. Under `Questions`, type `What accounts are available for this customer?`.
4. Under `Tasks`, type `Create a product application and notify the branch manager.`
5. Under `Triggers`, type `When a new service request is created, send an acknowledgment.`

Step 2 — Classify the agent experience

1. Next to `What accounts are available for this customer?`, label the scenario `Conversational agent`.
2. Next to `Create a product application and notify the branch manager.`, label the scenario `Conversational agent with actions`.
3. Next to `When a new service request is created, send an acknowledgment.`, label the scenario `Autonomous agent`.
4. Compare your labels with the facilitator explanation and update your notes if needed.

Step 3 — Map the role of the model

1. In your notes, add a fourth heading named `LLM`.
2. Type this sentence under `LLM`: `The model interprets intent, drafts language, and decides what to ask next.`
3. Add a fifth heading named `RAG`.
4. Type this sentence under `RAG`: `The agent retrieves trusted content before it answers so the response is grounded in current business data.`
5. Add a sixth heading named `Actions`.
6. Type this sentence under `Actions`: `Actions let the agent do work in systems such as SharePoint, Outlook, and Power Automate.`

Step 4 — Apply the concepts to the workshop flow

1. Open `https://copilotstudio.microsoft.com` in a browser tab.
2. On the home page, identify the **Create** entry point and the **Agents** area.
3. Say or write one example of a knowledge source you will use later, such as `Woodgrove Bank SharePoint site`.
4. Say or write one example of an action you will use later, such as `Create SharePoint item`.
5. Say or write one example of an autonomous trigger you will use later, such as `When a service request is created`.

Validation

1. Explain, in one sentence, the difference between a conversational agent and an autonomous agent.
2. Explain, in one sentence, why RAG is safer than relying on model memory alone for internal policy answers.
3. Identify one place in the Day 1 scenario where an action is required and one place where knowledge grounding is required.
4. If you can answer all three prompts without notes, you are ready for Lab 02.

Troubleshooting

> **Tip:** If the terms feel abstract, anchor them to the Woodgrove Bank examples: account lookup is knowledge, product application is an action, and service request acknowledgment is an autonomous trigger.

> **Tip:** If your group mixes up RAG and actions, remember that RAG helps the agent answer while actions help the agent do.

Facilitator Notes

1. Use the Woodgrove Bank story consistently so participants hear the same examples before they build them.
2. Pause after Step 2 and ask one [Maker], one [IT Pro], and one [Developer] participant to explain their classification.
3. Keep the pace brisk; this lab should create shared language, not become a deep AI theory session.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 02 — Copilot Studio Fundamentals

⏱️ Estimated time: 30 min

Overview

In this lab, you will tour the Copilot Studio interface and locate the four building blocks used throughout the workshop: instructions, knowledge, actions, and topics. You will also find the analytics area and the **Activity** page that you will later use to troubleshoot, pin important sessions, and improve your agent.

Copilot Studio agent canvas with tabs for overview topics tools and analytics

Prerequisites

1. Complete Lab 00 and confirm you can open Copilot Studio in the correct environment.
2. Complete Lab 01 so the terms `knowledge`, `actions`, `topics`, and `instructions` are familiar.
3. If you do not yet have an agent, be ready to create a temporary placeholder agent in this lab.

Step-by-Step Instructions

Step 1 — Open or create an agent for navigation practice

1. Go to `https://copilotstudio.microsoft.com` and verify the environment selector in the upper-right corner shows your workshop environment.
2. If you already have an agent, select **Agents** in the left navigation and open it.
3. If you do not have an agent, select **Create**, choose **Custom agent**, enter `Navigation Tour Agent` in the **Name** field, and select **Create**.
4. Wait until the agent overview page loads before continuing.

Step 2 — Find Instructions from the overview page

1. On the agent overview page, stay on the **Overview** tab.
2. In the **Details** section, locate the description and instructions text areas.
3. Select **Edit** if the section is read-only and confirm where you would update the agent purpose, response style, and boundaries.

4. Select **Cancel** or leave the values unchanged after you confirm the location.

Step 3 — Find Knowledge without relying on a screenshot

1. Stay on the **Overview** tab and scroll until you see the **Knowledge** section.
2. Confirm the **+ Add knowledge** button is visible.
3. Select **+ Add knowledge** and review the options such as **Public websites**, **SharePoint**, and **Documents**.
4. Close the panel without adding anything if you are using a temporary agent.
5. Say out loud or note down that knowledge sources are configured from the **Knowledge** section on the **Overview** tab.

Step 4 — Find Actions and tools without relying on a screenshot

1. In the top tab row, look for **Tools**.
2. If the **Tools** tab is not visible, select the overflow button such as **+ more** or the numeric tab expander and choose **Tools**.
3. On the **Tools** page, locate the **+ Add a tool** button.
4. Select **+ Add a tool** and confirm you can see choices such as **Connector**, **Prompt**, or **Flow** depending on your tenant capabilities.
5. Close the tool picker after you confirm the location.

Step 5 — Find Topics without relying on a screenshot

1. In the top tab row, select **Topics**.
2. If **Topics** is hidden, open the overflow control and choose **Topics**.
3. On the **Topics** page, locate the filters for **Custom** and **System** topics.
4. Select **+ Add a topic** and confirm where **From blank** or **From description** appears.
5. Close the create dialog so you can continue the tour.

Step 6 — Find Analytics and the Activity page

1. In the top tab row, select **Analytics**.
2. If **Analytics** is hidden, use the overflow control and choose **Analytics**.
3. Review the page sections that show conversation volume, resolution signals, and activity details.
4. Open the **Activity** page or the closest equivalent diagnostics page available in your tenant.
5. Confirm where you would inspect a single session, pin it for follow-up, and submit diagnostic feedback if needed.
6. If your tenant has not collected data yet, note that the page may show empty charts; the location is still correct.
7. Return to the **Overview** tab when you finish.

> **Tip:** Many Copilot Studio pages hide secondary tabs behind an overflow menu when the browser zoom level is high. If you lose **Topics**, **Tools**, or **Analytics**, reduce the browser zoom to 90% or 80% and check again.

Validation

1. Starting from the **Overview** tab, point to the exact location of **Knowledge** and say how to add a SharePoint source.
2. Starting from the top tab row, point to the exact location of **Topics** and say how to create a blank topic.
3. Starting from the top tab row, point to the exact location of **Tools** and say how to add a connector action.
4. Starting from the top tab row, point to the exact location of **Analytics** and say what type of data you expect to review there.
5. Starting from the diagnostics area, point to where you would inspect one conversation in the **Activity** tab.

Troubleshooting

> **Tip:** If the tab row changes after you resize the browser, use the tab overflow control rather than refreshing the page.

> **Tip:** If you cannot see **Analytics**, confirm the agent has finished provisioning and that you are not still on the home page.

> **Warning:** If Copilot Studio creates your temporary agent in the wrong environment, delete it and switch environments before recreating it.

Facilitator Notes

1. Demonstrate the interface once slowly and then ask participants to repeat the navigation themselves.
2. Call out that the UI can move tabs into overflow; this prevents unnecessary “my screen looks different” interruptions later.

3. Point out the **Activity** tab explicitly so later troubleshooting exercises feel familiar instead of hidden.
4. If the group already has agents, use the same agent throughout the tour to reduce setup noise.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 03 — Declarative Agents

⏱️ Estimated time: 60 min

Overview

In this lab, you will create a declarative agent for Microsoft 365 Copilot and Microsoft Teams. You will define the agent purpose, add a reusable prompt tool, test the orchestration, and publish the experience so participants can see how declarative agents extend Microsoft 365 Copilot with focused skills.

Declarative agent setup screen with name instructions and starter prompts

Prerequisites

1. Complete Lab 00 and confirm your environment is ready.
2. Complete Lab 02 so you can navigate to **Overview**, **Tools**, and **Publish**.
3. [Maker] Confirm you have permission to create agents in Copilot Studio.
4. [IT Pro] If participants will test in Microsoft Teams, confirm the tenant allows Teams app usage.

> **Note:** This lab creates a declarative agent for Microsoft 365 Copilot, which requires a **Microsoft 365 Copilot license** (such as the Copilot add-on for Microsoft 365 E3/E5). If your workshop account does not have this license, your facilitator will demonstrate this lab. You can observe the pattern without hands-on access.

Step-by-Step Instructions

Step 1 — Create a declarative agent shell

1. Go to `https://copilotstudio.microsoft.com` and select **Agents** in the left navigation.
2. Select **Microsoft 365 Copilot** from the agents list, then select **+ Add** on the Overview page.
3. In the **Name** field, enter `Woodgrove Banking Assistant`.
4. In the **Description** field, enter `Provides concise, step-by-step banking policy with empathy, encouragement, and interactive feedback for common IT issues.`

5. In the **Instructions** field, paste the sample text below and then select **Create**.

```
- Advise on common banking policies, account types, loan products, and regulatory co
- Respond with short numbered steps.
- Ask one focused follow-up question if the customer did not provide enough detail.
- Summarize the recommended action at the end of each answer.
- Stay professional, approachable, and supportive.
- Do not answer unrelated creative requests.
- Never share specific account balances, SSNs, or bypasses to compliance controls.
```

Step 2 — Add starter prompts participants can reuse

1. In the **Starter prompts** area, add a prompt titled `Account Types` with the text `What types of savings accounts does the bank offer?`.
2. Add a prompt titled `Loan Products` with the text `Can you explain the different mortgage options available?`
3. Add a prompt titled `Banking Terms` with the text `Can you explain what an APR is and how it affects my loan?`.
4. Add a prompt titled `Wire Transfer Policy` with the text `What is the policy for international wire transfers?`.
5. Add a prompt titled `Fraud Prevention` with the text `What should I do if I suspect fraudulent activity on my account?`.

Step 3 — Add a prompt tool

1. After the agent finishes provisioning, open the **Tools** tab.
2. Select **+ Add a tool**, then choose **Prompt**.
3. In the **Name** field, enter `Banking Policy Advisor`.
4. In the prompt authoring experience, choose **Manual instructions** and paste the sample prompt below.

```
I want you to act as an Banking Policy Advisor. I will provide the problem descripti
```

5. In the input area, add a **Text** input named `Question` and enter `What are the eligibility requirements for a home equity line of credit?` as the sample value.
6. Select **Test** to preview the prompt output.
7. Select **Save**, then select **Add and configure**.

Step 4 — Update the agent instructions to call the prompt tool

1. Return to the **Overview** tab.
2. In the **Details** section, select **Edit**.

3. Replace the instructions with the text below, then select **Save**.

```
When a user asks a banking policy question about accounts, loans, compliance, or fin
```

Step 5 — Test the declarative agent in Copilot Studio

1. In the **Test** pane, select **Start new test session** to begin with a clean conversation.
2. Enter `What are the eligibility requirements for a home equity line of credit?` in the message box and press **Enter**.
3. Review the response and confirm the answer is structured, short, and focused on banking policy.
4. Select one of the **Starter prompts** and confirm the same instruction pattern is followed.

Step 6 — Publish and test in Microsoft 365 Copilot or Teams

1. Select **Publish** in the top-right corner.
2. In the publish pane, review the **Channels** section and confirm **Microsoft 365 Copilot** and **Microsoft Teams** are listed.
3. Update the **Short description** to `Woodgrove Bank assistant for fast banking policy and account guidance.`
4. Update the **Developer name** field with your name or team name.
5. Select **Publish** and wait for the completion message.
6. Select **Copy link** or **See agent in Teams** and open the published agent in the chosen client.
7. In Microsoft 365 Copilot or Teams, enter `Can you explain the requirements for opening a business checking account?` and review the response.

> **Tip:** If you have access to Microsoft 365 Copilot developer mode, enter `-developer on` in Copilot Chat before testing so you can inspect which tool was selected at runtime.

Published declarative agent shown inside Microsoft 365 Copilot

Validation

1. Confirm the agent name shown in Copilot Studio is `Woodgrove Banking Assistant`.
2. Confirm the **Tools** tab lists the `Banking Policy Advisor` prompt.
3. Confirm the published experience answers at least one starter prompt and one free-form banking question.
4. If developer mode is available, confirm the `Banking Policy Advisor` prompt appears in the executed action details.

Troubleshooting

> **Tip:** If the prompt does not appear in the **Tools** list, return to the prompt authoring screen and make sure you selected **Save** before **Add and configure**.

> **Tip:** If the response ignores your prompt, reopen the **Details** section and verify the tool name in the instructions matches the saved tool name exactly.

> **Warning:** If publishing is blocked in a trial environment, finish the creation and validation steps in Copilot Studio and ask the facilitator to demo the published experience from a paid environment.

Facilitator Notes

1. Encourage participants to keep the declarative agent focused; the point is to extend Microsoft 365 Copilot with a narrow banking capability.
2. If the group is mixed, call out that [Maker] tasks happen in Copilot Studio while [IT Pro] participants should pay attention to channel and publishing constraints.
3. Keep the custom prompt lightweight because later labs introduce knowledge sources, topics, adaptive cards, and automation.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 04 — Solutions

⏱️ Estimated time: 45 min

Overview

In this lab, you will create a publisher and a solution for the Woodgrove Bank workshop assets, then validate that the solution can move between environments. The solution becomes the transport container for customizations, and the export/import exercise gives participants a concrete ALM checkpoint before later labs add more components.

Solution explorer showing Woodgrove Customer Service Agent solution

Prerequisites

1. Complete Lab 00 and confirm you are in the correct developer environment.
2. [Maker] Confirm you have the **Environment Maker**, **System Customizer**, or **System Administrator** role.
3. [IT Pro] Confirm you have a second environment available for import validation, or arrange a facilitator-provided validation environment.

Step-by-Step Instructions

Step 1 — Open Solution Explorer

1. Go to `https://copilotstudio.microsoft.com`.
2. In the left navigation, select **...** (More) and then select **Solutions**.
3. Wait for the **Solutions** page to load and confirm the correct environment name appears near the top of the page.

Step 2 — Create a publisher

1. Select **+ New solution**.
2. In the **Publisher** field, select **+ New publisher**.
3. In the **Display name** field, enter `Woodgrove Bank Solutions`.
4. In the **Name** field, enter `WoodgroveBankSolutions`.

5. In the **Description** field, enter `Copilot Studio workshop publisher for Woodgrove Bank assets`.
6. In the **Prefix** field, enter `cts`.
7. In the **Choice value prefix** field, round the suggested number down to the nearest thousand.
8. Select **Save**.

Step 3 — Create the workshop solution

1. Back in the **New solution** pane, confirm `Woodgrove Bank Solutions` is selected in the **Publisher** field.
2. In the **Display name** field, enter `Woodgrove Customer Service Agent`.
3. In the **Name** field, enter `WoodgroveCustomerServiceAgent`.
4. Leave the **Version** field at `1.0.0.0`.
5. Select the **Set as your preferred solution** checkbox. If this option does not appear in Copilot Studio, complete the creation here and then open `https://make.powerapps.com`, navigate to the solution, and set it as preferred from there.
6. Select **Create**.
7. When the new solution opens, confirm the header shows `Woodgrove Customer Service Agent`.

Step 4 — Add one transportable component for validation

1. Inside the solution, select **+ New** and choose **Environment variable**.
2. In the **Display name** field, enter `Woodgrove Scenario Name`.
3. In the **Name** field, accept the generated name that starts with `cts_`.
4. Set **Data type** to **Text**.
5. In the **Default value** field, enter `Woodgrove Bank Customer Service`.
6. Select **Save** and confirm the environment variable appears in the solution component list.

Step 5 — Export the solution

1. Select **Back** to return to the main **Solutions** list if the solution detail view is still open.
2. Select the row for **Woodgrove Customer Service Agent**.
3. Select **Export**.
4. Choose **Managed** for the package type if you are validating deployment to another environment, or **Unmanaged** if the facilitator instructed you to keep editing after import.
5. Confirm the **Version** is still `1.0.0.0` and start the export.
6. When the export finishes, download the `.zip` package to a known folder such as **Downloads**.

Step 6 — Import the solution into a validation environment

1. Switch the environment selector to the target validation environment.
2. Open **Solutions** again and select **Import solution**.
3. Upload the `.zip` file you exported in Step 5.
4. Review the import summary and select **Import**.
5. Wait for the import job to finish and then open the imported **Woodgrove Customer Service Agent** solution.
6. Confirm the **Woodgrove Scenario Name** environment variable is present.

> **Tip:** If your team has only one environment, ask the facilitator for a shared validation environment rather than importing back into the source environment.

Validation

1. In the source environment, confirm the solution list contains **Woodgrove Customer Service Agent** and that the **Current preferred solution** indicator points to it.
2. Open the exported `.zip` file location and confirm the package was downloaded successfully.
3. In the target environment, open the imported solution and confirm the `Woodgrove Scenario Name` environment variable exists.
4. Confirm the imported solution version is `1.0.0.0`.
5. State clearly whether you exported a **Managed** or **Unmanaged** solution and why.

Troubleshooting

> **Tip:** If **Solutions** is missing from the navigation, confirm your security role includes solution management rights in the current environment.

> **Tip:** If export fails because of pending dependencies, open the solution details and review whether a required component or connection reference is missing.

> **Warning:** Do not import the managed package back into the same source environment where you plan to continue editing the unmanaged version.

Facilitator Notes

1. Emphasize that the solution is the ALM boundary for the workshop, even when participants are still working in a trial environment.
2. The added environment variable is intentional; it gives participants a concrete, low-risk component to validate export and import before later labs add agents and flows.
3. If time is tight, demo the import step once and let participants complete the validation checklist rather than waiting for every import job live.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 05 — Prebuilt Agents

⏱️ Estimated time: 30 min

Overview

In this lab, you will deploy a Microsoft-provided agent template and adapt it to the workshop scenario. Prebuilt agents help teams accelerate delivery, understand product patterns, and prototype faster than starting from a blank canvas.

Agent template gallery with Safe Travels highlighted

Prerequisites

1. Complete Lab 00 and Lab 02.
2. Confirm you are signed in to Copilot Studio in the correct environment.
3. If you completed Lab 04, confirm `Woodgrove Customer Service Agent` is still the preferred solution.

Step-by-Step Instructions

Step 1 — Open the template gallery

1. Go to `https://copilotstudio.microsoft.com`.
2. On the home page, select **Create**.
3. Scroll to the **Start with an agent template** section.
4. Find the **Safe Travels** template and select it.

Step 2 — Review the template before you create it

1. On the template setup page, review the prefilled **Description**, **Instructions**, and **Knowledge** sections.
2. Note which pieces are already configured so you can compare them with the custom agent you build later.
3. Select **Create**.

Step 3 — Customize the prebuilt agent

1. After the agent provisions, go to the **Overview** tab.
2. In the **Knowledge** section, select **+ Add knowledge**.
3. Choose **Public websites**.
4. In the website field, enter `https://european-union.europa.eu/` and select **Add**.
5. Select **Add to agent**.
6. Review the updated knowledge list and confirm the public site now appears alongside the template content.

Step 4 — Test the prebuilt experience

1. Open the **Test** pane.
2. Enter `Do I need a visa to travel from the US to Amsterdam?` and press **Enter**.
3. Enter `How long does it take to get a US passport?` and review the response.
4. If the tenant allows activity details, open the **Activity map** to inspect how the answer was grounded.

Step 5 — Publish the template agent

1. Select **Publish** in the top-right corner.
2. Review the publish dialog and select **Publish** again.
3. If the environment supports channels, note that you can later add **Teams and Microsoft 365** from the **Channels** area.

> **Tip:** Template agents are useful learning assets even if you never deploy them to production. Use them to study naming patterns, starter prompts, and tool choices.

> **Note:** The Safe Travels agent is an exploration exercise and does not need to be added to the `Woodgrove Customer Service Agent` solution. Later labs (07–11) work exclusively with the custom agent you create in Lab 06.

Validation

1. Confirm the **Safe Travels** agent opens successfully.
2. Confirm the **Knowledge** section includes `https://european-union.europa.eu/`.
3. Confirm the test pane returns at least one grounded answer about travel.
4. Confirm you can reach the **Publish** action without navigation help.

Troubleshooting

> **Tip:** If the template gallery does not show **Safe Travels**, refresh the home page and confirm you are in a Copilot Studio-enabled environment.

> **Warning:** If test responses are vague, confirm the website knowledge source finished indexing before retesting.

Facilitator Notes

1. Use this lab to show the tradeoff between speed and control: templates are fast, but custom agents offer tighter fit.
2. Encourage participants to study what Microsoft preconfigures, especially starter prompts and instructions.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 06 — Build the Customer Service Agent

⏱️ Estimated time: 75 min

> **Model guidance:** See Model Baseline for current preferred and fallback models.

Overview

In this lab, you will create the main workshop agent, `Woodgrove Customer Service Agent`, by describing the experience in natural language. You will refine the generated result, add trusted public knowledge, connect the internal SharePoint site created in Lab 00, apply a metadata filter to improve retrieval quality, upload files for focused local references, and test grounded answers.

Custom agent created from natural language prompt in Copilot Studio

Prerequisites

1. Complete Lab 00 and Lab 04.
2. Confirm the **Woodgrove Bank** SharePoint site exists and you copied the site URL.
3. Confirm the **Woodgrove Customer Service Agent** solution exists and is set as the preferred solution.
4. [Maker] Confirm you can add knowledge sources in Copilot Studio.
5. [Maker] Have two short facilitator-provided banking reference files available locally, such as an account opening guide and a fee schedule document.

Step-by-Step Instructions

Step 1 — Create the agent from a natural language description

1. Go to `https://copilotstudio.microsoft.com` and select **Home**.
2. In the natural language create box, paste the sample prompt below.
3. Open the **Settings** gear if available and verify the **Preferred solution** is `Woodgrove Customer Service Agent`.
4. Submit the prompt and wait for the agent to provision.

```
You are the Woodgrove Customer Service Agent. You help bank employees answer custome

For customer inquiries:
1) Ask one focused question if details are missing.
2) Look up relevant account or service request information first.
3) Provide clear, numbered step-by-step guidance.
4) Offer one or two alternative paths if the first response does not resolve the inq
5) After two or three attempts, recommend escalation and provide a short case summar

For account lookups:
1) Ask what type of account or information the employee needs.

Never disclose sensitive financial data to unauthorized parties. Refuse requests to
```

Step 2 — Rename, set the model, and review the generated agent

1. After provisioning finishes, stay on the **Overview** tab.
2. In the **Details** section, select **Edit**.
3. Replace the generated **Name** with `Woodgrove Customer Service Agent`.
4. Review the generated description and instructions, then select **Save**.
5. In the **Select your agent's model** section, open the dropdown and select **GPT-5 Chat** if it is available in your region. This is the workshop baseline model. The picker labels **GPT-4.1** as "Default" — you need to actively select GPT-5 Chat. If GPT-5 Chat is not available, keep **GPT-4.1** and note the limitation in your workshop notes. GPT-4.1 is a fully supported generally available model that produces equivalent results for all Day 1 exercises — you do not need GPT-5 Chat to complete this workshop successfully.
6. Scroll through the Overview tab and check the top tab row for AI-suggested sections such as Knowledge, Tools, and Topics so you understand what Copilot proposed automatically.

Step 3 — Add public knowledge sources and disable open web search

1. In the **Knowledge** section, select **+ Add knowledge** > **Public websites** and enter the URL `https://support.microsoft.com`.
2. Add a second website with the value `https://learn.microsoft.com/troubleshoot/`.
3. Select **Add to agent**.
4. Open **Settings** and set **Use information from the Web** to **Off** so the agent uses only the sources you explicitly approved. This toggle may also appear on the **Overview** page depending on your agent configuration.
5. Also set **Allow the AI to use its own general knowledge** to **Off** so the agent relies exclusively on the knowledge sources you configure.

Step 4 — Add the internal SharePoint knowledge source and filter it

1. In the **Knowledge** section, select **+ Add knowledge** and choose **SharePoint**.
2. Paste the **Woodgrove Bank** site URL from Lab 00 into the **SharePoint URL** field.
3. In the **Name** field, enter `Woodgrove Bank`.
4. Select **Add** and then select **Add to agent**.
5. Open the newly added SharePoint source and select the **Advanced settings** tab.
6. Under **Filter this knowledge source**, select an attribute from the dropdown, choose an operator such as `is equal to` or `contains`, and enter a value. For example, use a filename filter such as `File name contains Account` or a scope rule such as `Modified date is within the last 365 days`.
7. Select **Save** and wait for the SharePoint source to finish connecting before you test the agent.

Knowledge section with public websites and SharePoint source added

Step 5 — Add file-based knowledge for focused local references

1. In the **Knowledge** section, select **+ Add knowledge** and choose **Files**.
2. Upload at least two facilitator-provided local files, such as an account opening guide and a fee schedule document.
3. Select **Add** and wait for the files to finish indexing.

> **Tip:** File-based knowledge is useful for focused operational references that are not stored in SharePoint or a public website.

Step 6 — Test grounded responses

1. In the **Test** pane, select **New test session**.
2. Enter `What checking accounts are currently active?` and review the answer.
3. Enter `Tell me about Sarah Johnson's account.` and observe whether the agent references your connected sources or asks a clarifying question.
4. Enter `Are there any frozen accounts?` and confirm whether the answer can use the SharePoint data or uploaded reference files.
5. If the tenant shows the **Activity map**, inspect the knowledge calls and citations.

Step 7 — Tighten the instructions if the answer quality needs work

1. In the **Details** section, select **Edit**.
2. Add a line such as `When answering from SharePoint, cite the source in plain language.`
3. Add a line such as `Keep customer service answers under 10 numbered steps unless the user asks for more detail.`

4. Add a line such as `Prefer the most relevant filtered enterprise source before using broader knowledge.`
5. Select **Save** and rerun the same tests.

Validation

1. Confirm the agent name is `Woodgrove Customer Service Agent`.
2. Confirm the **Knowledge** section lists both public websites, the `Woodgrove Bank` SharePoint source, and uploaded reference files.
3. Confirm **Use information from the Web** is set to **Off** in **Settings**.
4. Confirm the SharePoint source has at least one saved metadata filter or scoped rule.
5. Confirm the test pane returns at least one grounded answer with a trusted source reference.

Troubleshooting

> **Tip:** If the SharePoint source shows an authentication prompt, sign in with the same account you used to create the site and then reopen the **Knowledge** section.

> **Tip:** If the SharePoint URL is rejected, copy the site root URL only, such as `https://woodgrovebank.sharepoint.com/sites/WoodgroveBank`, and do not paste a list-specific URL.

> **Tip:** If the SharePoint connection succeeds but answers do not appear, wait for indexing to complete and test again with a question that clearly matches content in the site.

> **Tip:** If you see `You don't have access` or `Connection not authorized`, verify that your account is a member of the SharePoint site and that the environment's DLP policy does not block the connector.

> **Warning:** If responses still come from the open web, confirm **Use information from the Web** is set to **Off** in **Settings** and no extra public sources were added accidentally.

Facilitator Notes

1. Remind participants that natural language creation accelerates setup, but human review is still required.
2. Use this lab to reinforce good grounding discipline: trusted sources on, open web off unless the use case explicitly requires it.
3. Call out that stronger enterprise grounding often improves relevance but can slightly increase retrieval latency as more scoped sources are evaluated.
4. The SharePoint troubleshooting section is intentionally detailed because this is the most common blocker in live workshops.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 07 — Topics and Triggers

⏱️ Estimated time: 60 min

Overview

In this lab, you will add a structured topic that helps bank employees look up active customer accounts from the SharePoint **Customer Accounts** list. You will define a clear trigger, capture an optional account type, use a connector action to retrieve items, and guide the user toward a follow-up service request flow.

Topic authoring canvas with account inquiry flow

Prerequisites

1. Complete Lab 06 and confirm `Woodgrove Customer Service Agent` exists.
2. Confirm the **Customer Accounts** list in SharePoint contains at least four items and at least three items have **AccountStatus** set to `Active`.
3. [Maker] Confirm you can add a connector tool from within a topic.

Step-by-Step Instructions

Step 1 — Create a blank topic

1. Open **Woodgrove Customer Service Agent** in Copilot Studio.
2. Select the **Topics** tab. If it is hidden, use the tab overflow control and choose **Topics**.
3. Select **+ Add a topic** and then select **From blank**.
4. In the **Name** field, enter `Account Inquiry`.
5. In the **Description** field, enter `This topic helps users find active customer accounts from the Woodgrove Bank SharePoint Customer Accounts list. Trigger phrases: show active accounts, look up customer accounts, find accounts, account inquiry.`

Step 2 — Add a topic input for account type

1. In the topic canvas, select **Details** in the top toolbar (next to Topic checker).
2. In the **Input** tab, select **Create a new variable**.

3. Enter `VarAccountType` in the **Name** field.
4. Set **Variable data type** to **Text**.
5. Leave **How will the agent fill this input?** set to **Dynamically fill with the best option**.
6. Enter `The type of account the user is asking about, such as checking, savings, money market, or certificate of deposit.` in the **Description** field.
7. Save the input and close the details pane.

Step 3 — Add the message and SharePoint retrieval action

1. Add a **Send a message** node after the trigger and enter `I'll check the current list of active customer accounts for you.`
2. Select the **+** under the message node, choose **Add a tool**, and then select the **Connector** tab.
3. Search for the SharePoint **Get items** action and select it.
4. Rename the tool to `Get active accounts`.
5. In **Site Address**, select your `Woodgrove Bank` SharePoint site.
6. In **List Name**, select `Customer Accounts`.
7. In **Filter Query**, enter `AccountStatus eq 'Active'`.

> **Tip:** SharePoint OData filters are case-sensitive for Choice column values. Confirm the exact spelling and casing of the `Active` choice value in your **Customer Accounts** list matches what you enter here. If you used `active` or `ACTIVE` when setting up Lab 00, the filter will silently return zero results.

8. Save the connector configuration.

Step 4 — Store the results and summarize them for the user

1. In the connector output settings, store the response in a variable named `VarAccounts`. Select the variable, open its properties, and change the scope from **Topic** to **Global** so later topics can access the results.
2. Add a **Send a message** node after the connector.
3. Switch the message to **Formula** mode if your tenant supports it.
4. Enter a formula such as `"Active accounts:" & Char(10) & Concat(Global.VarAccounts.value, "- " & Title & " | " & AccountNumber & " | " & AccountType & " | $" & CurrentBalance & Char(10))`. If the editor flags an error with unqualified column names, switch to `ThisRecord.Title`, `ThisRecord.AccountNumber`, etc.
5. Add an **Ask a question** node after the message.
6. Use the question `Would you like to view the details of a specific account?` and choose a **Yes/No** response type.

Step 5 — Save and reinforce topic routing

1. Select **Save** in the topic toolbar.
2. Return to the **Overview** tab and select **Edit** in the **Details** section.
3. Add this instruction on a new line: `When a user asks about active accounts, customer accounts, or account inquiries, use the Account Inquiry topic.`
4. Select **Save**.

Topic test pane showing active accounts and yes no question

Step 6 — Test the topic

1. Open the **Test** pane and select **Start new test session** for a clean conversation.
2. Enter `Show active accounts` and press **Enter**.
3. Confirm the topic returns an account list and the yes/no follow-up question.
4. Reply `No` to confirm the topic ends cleanly.

> **Tip:** The topic description is important. Generative orchestration uses it to decide when the topic should run, even if the user wording is slightly different from your example phrases.

Validation

1. Confirm the **Topics** list includes `Account Inquiry`.
2. Confirm the topic input `VarAccountType` exists in the topic details.
3. Confirm the SharePoint action is configured against the `Customer Accounts` list.
4. Confirm the test session returns active account records and asks whether the user wants to view details of a specific account.

Troubleshooting

> **Tip:** If the topic does not trigger, confirm the description clearly mentions accounts and inquiries, then add one or two example phrases in the trigger area if your tenant shows that option.

> **Tip:** If the SharePoint connector cannot find the list, refresh the page and reopen the action after confirming the site and list exist in SharePoint.

> **Tip:** If the formula fails, start with a static message, save the topic, and then add the formula again with a smaller expression.

Facilitator Notes

1. This is the first lab where the agent becomes structured rather than purely generative; slow down enough for participants to understand why the topic exists.
2. Keep the data retrieval simple. The goal is successful retrieval and routing, not perfect OData filtering by account type.
3. Call out the `Global.VarAccounts` variable because Lab 08 reuses it for the adaptive card choices.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 08 — Adaptive Cards

⏱️ Estimated time: 45 min

Overview

In this lab, you will replace a plain-text follow-up with an interactive Adaptive Card so users can submit a service request directly in the conversation. You will prototype the card visually, capture form values, and prepare the outputs for the automation you build in Lab 09.

Adaptive card shown in Copilot Studio topic designer

Prerequisites

1. Complete Lab 07 and confirm the `Account Inquiry` topic works.
2. Confirm `Global.VarAccounts` is being populated by the SharePoint **Get active accounts** action.
3. Open the Adaptive Card Designer at `https://adaptivecards.io/designer/` in a second browser tab.

Step-by-Step Instructions

Step 1 — Create the service request topic shell

1. Open **Woodgrove Customer Service Agent** and select the **Topics** tab.
2. Select **+ Add a topic** and choose **From blank**.
3. In the **Name** field, enter `Submit service request`.
4. In the **Description** field, enter `This topic collects a service request after the user says yes to submitting one from the account inquiry results.`
5. Save the topic shell if your tenant prompts you to do so.

Step 2 — Prototype the card in the external designer

1. In the browser tab for `https://adaptivecards.io/designer/`, select **New card**.
2. Add a **TextBlock** element and set the text to `Submit a Service Request`.
3. Add an **Input.Text** element with the label `Customer name`.

4. Add an **Input.ChoiceSet** element and label it `Request type`. Add three choices: `Dispute`, `Address Change`, and `Fee Reversal`.
5. Add an **Input.Text** element with the label `Description` and enable multiline mode.
6. Add a second **Input.ChoiceSet** element and label it `Priority`. Add three choices: `Low`, `Normal`, and `High`.
7. Add an **Action.Submit** button with the title `Submit request`.
8. Copy the JSON from the **Card Payload Editor**.

> **Tip:** The external designer is the fastest way to understand the card structure before you move the card into Copilot Studio.

Step 3 — Add the card to the topic

1. Return to Copilot Studio and open the `Submit service request` topic.
2. Select **+** on the canvas and choose **Ask with adaptive card**.
3. Click on the **Adaptive Card** node to open its properties panel on the right.
4. Select **Edit adaptive card** to open the built-in **Adaptive card designer**.
5. In the **Card payload editor** at the bottom, paste the JSON from the external designer (or type it directly).
6. Select **Save** and then **Close** to return to the topic canvas. The output variables become visible under the card node.

Step 4 — Build the card with form fields

1. Reopen the **Adaptive Card** node and select **Edit adaptive card**.
2. In the **Card payload editor**, click the **JSON card** dropdown at the top of the properties panel and switch to **Formula** mode.
3. Use the sample formula below as a starting point, then adjust field names if your SharePoint schema differs.

```
{
  type: "AdaptiveCard",
  version: "1.5",
  body: [
    { type: "TextBlock", text: "Submit a Service Request", weight: "Bolder", size: "l
    {
      type: "Input.Text",
      id: "customerNameId",
      label: "Customer name",
      isRequired: true,
      placeholder: "e.g. Sarah Johnson"
    },
```

```
      {
        type: "Input.ChoiceSet",
        id: "requestTypeId",
        label: "Request type",
        isRequired: true,
        choices: [
          { title: "Dispute", value: "Dispute" },
          { title: "Address Change", value: "Address Change" },
          { title: "Fee Reversal", value: "Fee Reversal" }
        ]
      },
      {
        type: "Input.Text",
        id: "descriptionId",
        label: "Description",
        isMultiline: true,
        placeholder: "Describe the service request details…"
      },
      {
        type: "Input.ChoiceSet",
        id: "priorityId",
        label: "Priority",
        isRequired: true,
        choices: [
          { title: "Low", value: "Low" },
          { title: "Normal", value: "Normal" },
          { title: "High", value: "High" }
        ]
      }
    ],
    actions: [
      { type: "Action.Submit", title: "Submit request" }
    ]
  }
```

4. Save the card and close the editor.
5. Confirm the output variables `customerNameId`, `requestTypeId`, `descriptionId`, and `priorityId` appear under the node.

Step 5 — Route users into the card from the previous topic

1. Return to the **Overview** tab and select **Edit** in the **Details** section.
2. Add the instruction `If the user answers yes when asked whether they want to submit a service request, use the Submit service request topic.`
3. Save the instructions.
4. Return to the `Account Inquiry` topic and confirm the yes/no question is still present.

Step 6 — Test the adaptive card experience

1. Open the **Test** pane and select **Start new test session**.
2. Enter `Show active accounts` and wait for the **Account Inquiry** topic to return results.
3. Reply `Yes`.
4. Confirm the **Submit service request** adaptive card appears.
5. Enter a customer name, select a request type, add a description, choose a priority, and submit the card.

Adaptive card in test pane with service request fields

Validation

1. Confirm the **Submit service request** topic exists in the **Topics** list.
2. Confirm the adaptive card node exposes `customerNameId`, `requestTypeId`, `descriptionId`, and `priorityId` outputs.
3. Confirm a test conversation can move from `Account Inquiry` into `Submit service request`.
4. Confirm the submitted card returns values instead of a blank response.

Troubleshooting

> **Tip:** If the adaptive card editor rejects the formula, save a static JSON version first and then introduce the dynamic expressions in small edits.

> **Tip:** If the card layout looks wrong, prototype again in the external designer at `https://adaptivecards.io/designer/` and then paste the cleaned payload back into Copilot Studio.

> **Tip:** If output variables do not appear, confirm the `id` property is set on each input element in the card JSON.

Facilitator Notes

1. Participants often understand cards faster when they see the external designer first and the in-product card editor second.
2. Reinforce that the card outputs are the bridge to automation in Lab 09.
3. If formula mode is unavailable in a tenant, use static placeholder choices for the live build and explain how the dynamic pattern works conceptually.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 09 — Agent Flows

⏱️ Estimated time: 45 min

Overview

In this lab, you will connect the Adaptive Card from Lab 08 to a Power Automate flow so the form submission creates a service request record in SharePoint and emails the service team. This turns the service request topic from a conversation into a working business process.

Power Automate flow designer with agent trigger and SharePoint actions

Prerequisites

1. Complete Lab 08 and confirm the adaptive card submits values for `customerNameId`, `requestTypeId`, `descriptionId`, and `priorityId`.
2. Confirm the `Customer Accounts` SharePoint list contains at least one item with **AccountStatus** set to `Active`.
3. [Maker] Confirm you can create flows in the same environment as the agent.
4. [IT Pro] Confirm the SharePoint and Office 365 Outlook connectors are allowed by DLP policy.

Step-by-Step Instructions

Step 1 — Create the SharePoint list that will store service requests

1. Open the `Woodgrove Bank` SharePoint site from Lab 00.
2. Select **New** > **List**.
3. Select **Blank list**.
4. In the **Name** field, enter `Service Requests` and select **Create**.
5. Add the following columns from **+ Add column**: `RequestType` as **Single line of text**, `RequestDescription` as **Multiple lines of text**, `Priority` as **Choice**, `Status` as **Choice**, and `SubmittedOn` as **Date and time**.
6. In the **Priority** choice column, add `Low`, `Normal`, and `High` as choices with `Normal` as the default.

7. In the **Status** choice column, add `New`, `In Progress`, and `Resolved` as choices with `New` as the default.

Step 2 — Add a flow after the Adaptive Card submission

1. Open **Woodgrove Customer Service Agent** and then open the `Submit service request` topic.
2. Under the **Ask with adaptive card** node, select **+**.
3. Choose **Add a tool** and then select **Agent flow** under **Create new**.
4. Wait for the Agent flow designer to open in the same window.

Step 3 — Define flow inputs from the Adaptive Card

1. Select the **When an agent calls the flow** trigger to expand it.
2. Add a **Text** input named `CustomerName`.
3. Add a **Text** input named `RequestType`.
4. Add a **Text** input named `RequestDescription`.
5. Add a **Text** input named `Priority`.

Step 4 — Create the service request record in SharePoint

1. Select the **+** button between the trigger and **Respond to the agent**.
2. Search for the SharePoint **Create item** action and add it.
3. Rename the step to `Create Service Request`.
4. In **Site Address**, select the `Woodgrove Bank` site.
5. In **List Name**, select `Service Requests`.
6. Map **Title** to the `CustomerName` trigger input. SharePoint requires a Title column — using the customer name makes the list view scannable.
7. Map **RequestType** to the `RequestType` trigger input.
8. Map **RequestDescription** to the `RequestDescription` trigger input.
9. Map **Priority** to the `Priority` trigger input.
10. Map **Status** to the value `New`.
11. Map **SubmittedOn** to `utcNow()` if the field is available. To enter this, select the **Expression** tab (not **Dynamic content**) in the field editor and type `utcNow()` exactly as shown, then select **OK**.

Step 5 — Notify the service team

1. Add an **Office 365 Outlook – Send an email (V2)** action after the **Create Service Request** step.
2. In **To**, enter a valid email address for the service team or your own test email.
3. In **Subject**, enter `New service request from` followed by the `CustomerName` input.

4. In **Body**, include the customer name, request type, description, and priority level.

Step 6 — Return a confirmation message to the topic

1. Add a **Respond to agent** action.
2. Add a **Text** output named `ConfirmationMessage`.
3. Set the output value to `Your service request has been submitted successfully. A team member will review it shortly.`
4. Save the flow and return to the topic.

Step 7 — Map the Adaptive Card outputs to the flow inputs

1. In the topic tool node, map `CustomerName` to `customerNameId` from the adaptive card output.
2. Map `RequestType` to `requestTypeId`.
3. Map `RequestDescription` to `descriptionId`.
4. Map `Priority` to `priorityId`.
5. Add a **Send a message** node after the flow.
6. Insert the `ConfirmationMessage` flow output into the message.
7. Save the topic.

Topic node showing flow inputs mapped from adaptive card outputs

Step 8 — Test the end-to-end process

1. Open the **Test** pane and start a new session.
2. Enter `Show active accounts` and move through the `Account Inquiry` topic.
3. Reply `Yes` to open the adaptive card.
4. Enter a customer name, select a request type, add a description, choose a priority, and submit the card.
5. Wait for the confirmation message from the flow.
6. Open the `Service Requests` SharePoint list and confirm a new item was created with Status set to `New`.

> **Tip:** The Adaptive Card submission is the trigger point. If the flow works but the topic does not, the issue is usually input mapping rather than the actions themselves.

Validation

1. Confirm the Power Automate flow contains the trigger, `Create Service Request`, `Send an email (V2)`, and `Respond to agent` steps.

2. Confirm the adaptive card outputs are mapped correctly in the topic tool node.
3. Confirm a new row appears in the `Service Requests` SharePoint list after a test submission with Status set to `New`.
4. Confirm the service team receives the email notification.

Troubleshooting

> **Tip:** If the flow runs but the topic returns no message, open the **Respond to agent** step and confirm the `ConfirmationMessage` output exists and is mapped in the topic.

> **Tip:** If the `CustomerName` input is blank, resubmit the adaptive card and verify the `Input.Text` id values match the expected output variable names.

> **Tip:** If the Priority or Status choice columns reject the flow values, open the SharePoint list settings and confirm the exact spelling and casing of each choice matches what the flow sends.

> **Warning:** If the `Service Requests` list does not exist before you save the flow, the SharePoint actions will not populate the expected column pickers.

Facilitator Notes

1. Call out explicitly that this lab connects conversational capture to system writeback, which is the turning point for business value.
2. If time is limited, provide a pre-created `Service Requests` list so participants can focus on the flow and topic mapping.
3. The user requirement for this workshop is satisfied only when the Adaptive Card submission writes back to SharePoint, so validate that step carefully.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 10 — Event Triggers

⏱️ Estimated time: 25 min

Overview

In this lab, you will make the Woodgrove Customer Service Agent respond autonomously when a new SharePoint service request is created. The event trigger will detect the list item, send service request details to the agent, and use an email action to acknowledge receipt without waiting for a user to start a chat.

Event trigger configuration for new SharePoint service request creation

Prerequisites

1. Complete Lab 06 and confirm the agent is working.
2. Confirm the `Service Requests` SharePoint list exists in the `Woodgrove Bank` site.
3. Confirm you can create flows or triggers in the environment.
4. [IT Pro] Confirm DLP policy allows SharePoint and Outlook connectors.

Step-by-Step Instructions

Step 1 — Confirm orchestration is enabled

1. Open **Woodgrove Customer Service Agent** in Copilot Studio.
2. Select **Settings**.
3. In the **Orchestration** section, confirm **Use generative AI orchestration for your agent's responses** is set to **Yes**.
4. Select **Save** if you changed the setting.

Step 2 — Create the SharePoint event trigger

1. Return to the **Overview** tab.
2. In the **Triggers** section, select **+ Add trigger**.
3. In the **Add trigger** dialog, select the **Featured** filter tab if it is not already selected.
4. Select **When an item is created** (SharePoint).

5. In **Trigger name**, enter `New Service Request Created in SharePoint`.
6. Wait for the connections to initialize and select **Next**.
7. In **Site Address**, select the `Woodgrove Bank` site.
8. In **List Name**, select `Service Requests`.
9. In **Additional instructions to the agent when it's invoked by the trigger**, paste the text below.

```
New Service Request Created in SharePoint: {Body}

Use the Acknowledge SharePoint service request tool to generate and send a confirmat
Do not wait for user input. Work autonomously.
```

> **Note:** `{Body}` in the text above is a static placeholder only. You will replace it with actual SharePoint field expressions in Step 3 when you edit this flow in Power Automate.

10. Select **Create trigger**.

Step 3 — Edit the trigger payload in Power Automate

1. In the **Triggers** section, open the **...** menu for `New Service Request Created in SharePoint`.
2. Select **Edit in Power Automate**.
3. Open the step that sends the prompt to the copilot.
4. Replace the default body with an expression that includes the submitter name, submitter email, title, description, priority, and ticket ID.
5. Use the **dynamic content** picker to insert the service request fields into the prompt body. If your trigger provides dynamic content tokens directly, select them from the picker. If you need an expression, use a concatenation similar to the sample below and then select **Save** or **Publish**.

> **Tip:** The exact expression path depends on how Copilot Studio wraps the SharePoint trigger. If the sample expression below returns null values, use the dynamic content picker instead of manual expressions to select the correct fields.

```
concat('Submitted By Name: ', triggerOutputs()?['body/Author/DisplayName'], '
Submitted By Email: ', triggerOutputs()?['body/Author/Email'], '
Title: ', triggerOutputs()?['body/Title'], '
Issue Description: ', triggerOutputs()?['body/Description'], '
Priority: ', triggerOutputs()?['body/Priority/Value'], '
Ticket ID: ', triggerOutputs()?['body/ID'])
```

Step 4 — Add the email action the trigger will call

1. Return to Copilot Studio and open the **Tools** tab.
2. Select **+ Add a tool** and choose **Connector**.
3. Search for **Send an email (V2)** from **Office 365 Outlook**.
4. Select **Add and configure**.
5. Set **Name** to `Acknowledge SharePoint service request`.
6. Set **Description** to `Sends an email acknowledgment that a SharePoint service request was received.`
7. Customize the **To** parameter so the description says `The email address of the person submitting the SharePoint service request`.
8. Customize the **Body** parameter so the description says `An acknowledgement that the service request was received and the team will respond within three working days.`
9. Save the tool.

Step 5 — Test the autonomous trigger

1. Go back to the **Overview** tab.
2. Select the **Test trigger** icon next to `New Service Request Created in SharePoint`.
3. In a second browser tab, open the `Service Requests` SharePoint list and select **New**.
4. Create a service request with **Title** set to `Unable to connect to VPN`, **Description** set to `Unable to connect after a recent password update`, and **Priority** set to `Normal`.
5. Save the SharePoint item.
6. Return to Copilot Studio, refresh the trigger test panel until the event appears, and select **Start testing**.
7. Allow connector prompts if the test panel asks for permission.
8. Review the activity output and confirm the tool call executes.
9. Check the submitter mailbox and confirm the acknowledgment email arrived.

Validation

1. Confirm the **Triggers** section lists `New Service Request Created in SharePoint`.
2. Confirm the trigger payload includes service request metadata rather than a generic body blob.
3. Confirm the `Acknowledge SharePoint service request` tool exists on the **Tools** tab.
4. Confirm a newly created SharePoint service request causes an autonomous test run.
5. Confirm the acknowledgment email is delivered.

Troubleshooting

> **Tip:** If the trigger never appears in the test panel, verify that the SharePoint item was created after the trigger was enabled and refresh the panel again after a short wait.

> **Tip:** If the email tool does not run, confirm the tool name in the trigger instructions matches the saved tool name exactly.

> **Tip:** If the trigger can read the service request but cannot send email, reconnect the Office 365 Outlook connector under the tool configuration.

> **Warning:** Event triggers use maker credentials. Review data access carefully before you publish the agent broadly.

Facilitator Notes

1. Use this lab to explain the difference between user-initiated topics and system-initiated triggers.
2. Call out the credential model clearly so participants understand the governance implications.
3. If testing is slow, demonstrate one successful trigger and let participants focus on reading the configuration rather than waiting for repeated runs.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 11 — Publish Agent

⏱️ Estimated time: 30 min

Overview

In this lab, you will publish the Woodgrove Customer Service Agent, add the Teams and Microsoft 365 Copilot channel, and validate that the latest version is available where users actually work. You will also review the broader channel landscape, including WhatsApp, so the workshop can end with a production-minded release discussion.

Publish dialog and Teams channel configuration for the agent

Prerequisites

1. Complete Labs 06 through 10 so the agent has meaningful capabilities to publish.
2. Confirm your environment supports publishing. Trial environments may require extra admin steps or may block the final publish action.
3. Confirm you can sign in to Microsoft Teams with the same workshop account.

Step-by-Step Instructions

Step 1 — Publish the latest agent version

1. Open **Woodgrove Customer Service Agent** in Copilot Studio.
2. Select **Publish** in the upper-right corner.
3. Review the confirmation dialog and select **Publish** again.
4. Wait for the completion notification at the top of the page.

Step 2 — Add the Teams and Microsoft 365 channel

1. Select the **Channels** tab in the top navigation.
2. Under **Microsoft channels**, select **Teams and Microsoft 365 Copilot**.
3. Select **Add channel**.
4. Wait for the green success notification.
5. Select **See agent in Teams** to open the installation experience in a new browser tab.

Step 3 — Add the agent to Teams for yourself

1. In the Teams installation page, select **Add**.
2. Wait for the app installation to finish.
3. Select **Open** to launch the agent in Microsoft Teams.
4. In Teams, pin the app if the option appears so you can find it easily during testing.

Step 4 — Review availability options for broader rollout

1. Return to the Copilot Studio browser tab.
2. In the **Teams and Microsoft 365** channel pane, select **Availability options**.
3. Review **Copy link**, **Show to my teammates and shared users**, and **Show to everyone in my org**.
4. If your tenant process allows it, select **Show to everyone in my org** and then select **Submit for admin approval**.
5. Note that a Teams administrator must publish the submitted app before it appears broadly in the app store.
6. Return to the main **Channels** page and confirm that **WhatsApp** appears in the **Other channels** section for later rollout planning, even though this workshop does not configure it hands-on.

> **Tip:** If the **Other channels** section shows channels as unavailable, check the banner at the top of the Channels page. When the agent uses **Microsoft authentication**, only Teams, Microsoft 365, and SharePoint channels are active. To enable other channels, select **change your authentication settings** in the banner.

> **Warning:** If you are using a trial environment that blocks publishing, complete the page review and validation steps with a facilitator demo or a paid environment. Do not spend workshop time repeatedly retrying a blocked publish action.

Agent opened inside Microsoft Teams after successful install

Validation

1. Confirm the Copilot Studio **Publish** notification indicates the latest version is live.
2. Confirm the **Channels** tab shows **Teams and Microsoft 365 Copilot** under **Microsoft channels**.
3. Confirm you can install the app in Microsoft Teams and open it successfully.

4. In Teams, send a validation prompt such as `I need help with a VPN issue` and confirm the agent responds.
5. Confirm the Teams publishing path is clear by identifying whether your app is self-installed, shared with teammates, or submitted for admin approval.
6. Confirm you can identify WhatsApp as an additional supported publish channel for future rollout planning.

Troubleshooting

> **Tip:** If the **Channels** tab is hidden, use the top tab overflow control to locate it.

> **Tip:** If Teams opens but the app does not install, sign out and back in to Teams with the same account you used in Copilot Studio.

> **Tip:** If your org-wide submission stays pending, ask the Teams administrator to review the app in **Teams Admin Center** > **Apps**.

Facilitator Notes

1. Participants often think publishing and channel configuration are the same task. Demonstrate the difference explicitly.
2. Keep the validation focused on Teams because the user requirement for this workshop specifically calls for Teams publishing validation.
3. Mention WhatsApp briefly as a GA channel so participants see that channel strategy is broader than Teams alone.
4. If the environment cannot publish, provide screenshots or a live tenant demo so participants still see the end state.

Copilot Studio Workshop

Day 1 — Foundation Track

Lab 12 — Licensing

⏱️ Estimated time: 20 min

Overview

This lab is presentation-first and closes the day with the commercial model behind Copilot Studio. You will review Copilot Credits, understand when Microsoft 365 Copilot user licensing is enough, identify when Copilot Studio capacity is still consumed, and answer common customer questions about trials, production rollout, and cost planning.

Licensing decision slide with credits packs pay as you go and Microsoft 365 Copilot

Prerequisites

1. Complete the earlier Day 1 labs so you have concrete examples of knowledge, actions, topics, flows, and publishing.
2. Open a note-taking app for the scenario answers in this lab.
3. [IT Pro] Be ready to capture follow-up questions about tenant capacity, billing ownership, and governance.

Step-by-Step Instructions

Step 1 — Record the core licensing terms

1. In your notes, create five headings: `Copilot Credits`, `Capacity packs`, `Pay-as-you-go`, `Copilot Credits Pre-Purchase Plan`, and `User licenses`.
2. Under `Copilot Credits`, write `Usage currency for work performed by Copilot Studio agents.`
3. Under `Capacity packs`, write `Prepaid monthly credit capacity pooled at the tenant level.`
4. Under `Pay-as-you-go`, write `Azure-billed usage for actual credits consumed.`
5. Under `Copilot Credits Pre-Purchase Plan`, write `A one-year prepaid option purchased through the Azure portal. Copilot Credit Commit Units (CCCUs) automatically cover Copilot Credit consumption across eligible products.`

6. Under `User licenses`, write `Makers need Copilot Studio user access, and end users may still trigger Copilot Studio credits when agents perform advanced work.`

Step 2 — Map the workshop scenarios to licensing behavior

1. Under a new heading named `Scenario mapping`, list `Basic internal Q&A in Microsoft 365 Copilot` and mark it `Often covered by user license for basic interaction`.
2. List `Product application with SharePoint writeback and branch manager email` and mark it `Consumes Copilot Studio credits`.
3. List `Autonomous service request acknowledgment trigger` and mark it `Consumes Copilot Studio credits`.
4. List `Maker building and editing agents` and mark it `Requires Copilot Studio maker access`.

Step 3 — Review the planning checklist

1. Open a browser tab and go to `https://aka.ms/copilotstudioestimator`.
2. Review the estimator landing page and note where you would model projected workload.
3. In your notes, list three planning actions: `Estimate usage`, `Disable unused tools`, and `Monitor capacity in admin center`.
4. Add a fourth action: `Mix prepaid capacity with pay-as-you-go for burst protection when appropriate`.

Step 4 — Review Savings analytics

1. If your environment already has analytics data, open the agent **Analytics** tab and locate the **Savings** section in the upper right.
2. Select **Calculate savings** to see how the agent tracks time and money saved, or review the section description if no data is available yet.
3. If the Savings section is not populated in your environment, use the facilitator demo or screenshot for this section.
4. In your notes, add three metrics leadership would care about: `Time saved`, `Cost avoided`, and `Adoption by business process`.
5. Add one sentence that says `Savings analytics matter because they connect agent behavior to measurable business value, not only technical usage.`

Step 5 — Work through the FAQ prompts

1. Write the question `Do Microsoft 365 Copilot user licenses remove the need for Copilot Studio credits?` and answer `No, advanced actions, triggers, connectors, and external publishing can still consume Copilot Studio credits.`

2. Write the question `What happens when a trial expires?` and answer `Authoring and testing access may end or be reduced, and published experiences should be moved to a paid capacity model before production use.`
3. Write the question `What is the safest production planning habit?` and answer `Estimate first, monitor after launch, and review capacity regularly.`
4. Write the question `Who needs a maker license or maker access?` and answer `Anyone who creates or manages agents in Copilot Studio.`

> **Note:** Trial environments are ideal for workshops and proof of concept work, but they are not a durable production strategy. Plan the move to paid capacity before the trial clock becomes a delivery risk.

Step 6 — Summarize your recommendation for Woodgrove Bank

1. In your notes, write a two-sentence recommendation for the Woodgrove Bank scenario.
2. Include one sentence on why the product application and event trigger workflows need Copilot Studio capacity planning.
3. Include one sentence on why monitoring after publish, including ROI analytics, is part of the implementation and not an afterthought.

Validation

1. Explain the difference between pay-as-you-go and capacity packs in one sentence each.
2. Identify which Day 1 lab first introduced an experience that clearly consumes Copilot Studio credits beyond a basic chat answer.
3. Explain what happens when the trial expires and what the customer should do before that date.
4. Confirm you can answer the FAQ question about Microsoft 365 Copilot user licenses versus Copilot Studio credits.
5. Explain why Savings analytics is valuable for leadership conversations about adoption and investment.

Troubleshooting

> **Tip:** If participants think `chat equals free`, bring them back to the Day 1 examples where the agent processed applications, ran flows, and sent email.

> **Tip:** If the group confuses maker access with end-user access, separate the conversation into

> two lists: `who builds` and `who uses`.

> **Warning:** Avoid promising exact pricing outcomes during the workshop unless you are using current customer-approved numbers and the official estimator.

Day 1 Exit Checklist (Day 2 readiness gate)

Before starting Day 2, confirm each item. If three or more are missing, plan to observe Day 2 rather than build.

- ☐ Your Woodgrove Customer Service Agent is published and responds to a test query
- ☐ You can identify which model your agent uses and switch it (preferred GPT-5 Chat, fallback GPT-4.1 — see facilitator-guide/model-baseline.md)
- ☐ You added at least one knowledge source and observed grounded vs ungrounded behavior
- ☐ You created at least one custom topic (Lab 07) end-to-end
- ☐ You wired one adaptive card and one Power Automate flow into a topic (Labs 08-09)
- ☐ You understand which environment your agent lives in and your role (Maker vs Owner)
- ☐ You have access to a Day 2 environment (or know which provided one to use)
- ☐ Your tenant has Copilot Studio credits or messages allocated (or you know who can grant them)

Facilitator Notes

1. Keep this lab anchored to the workshop scenario, not abstract licensing theory.
2. Use the FAQ answers as discussion prompts and let [IT Pro] participants add tenant governance considerations.
3. Spend five focused minutes on Savings analytics because this is often the bridge from technical success to leadership buy-in.
4. End by reminding participants that technical success and commercial readiness are both required for a credible customer deployment.